GenAI

Generative AI

↓

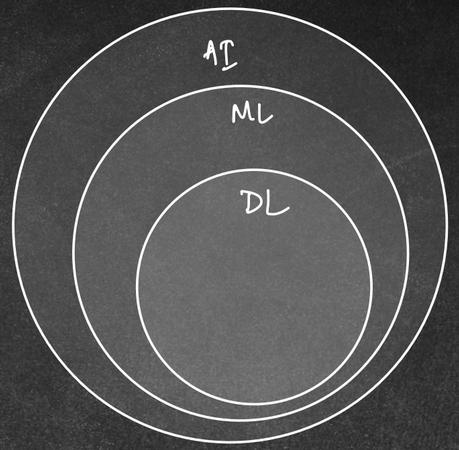
create new content

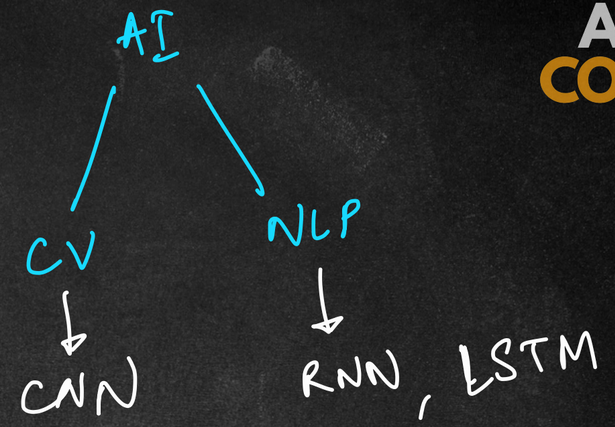
ANN

RNN → LSTM

CNN

Transformer





APNA COLLEGE

images/videos → dalle., midjourney

text → chatgpt, claude, gemini

audio/sound/music → sound draw

code → copilot

⋮

LLMs

Large Language Models

- ✓ Billions of parameters
- class of models in NLP
- ✓ trained on huge datasets
- ✓ capabilities ↑↑

How are you?

Q: why should I code in Python?

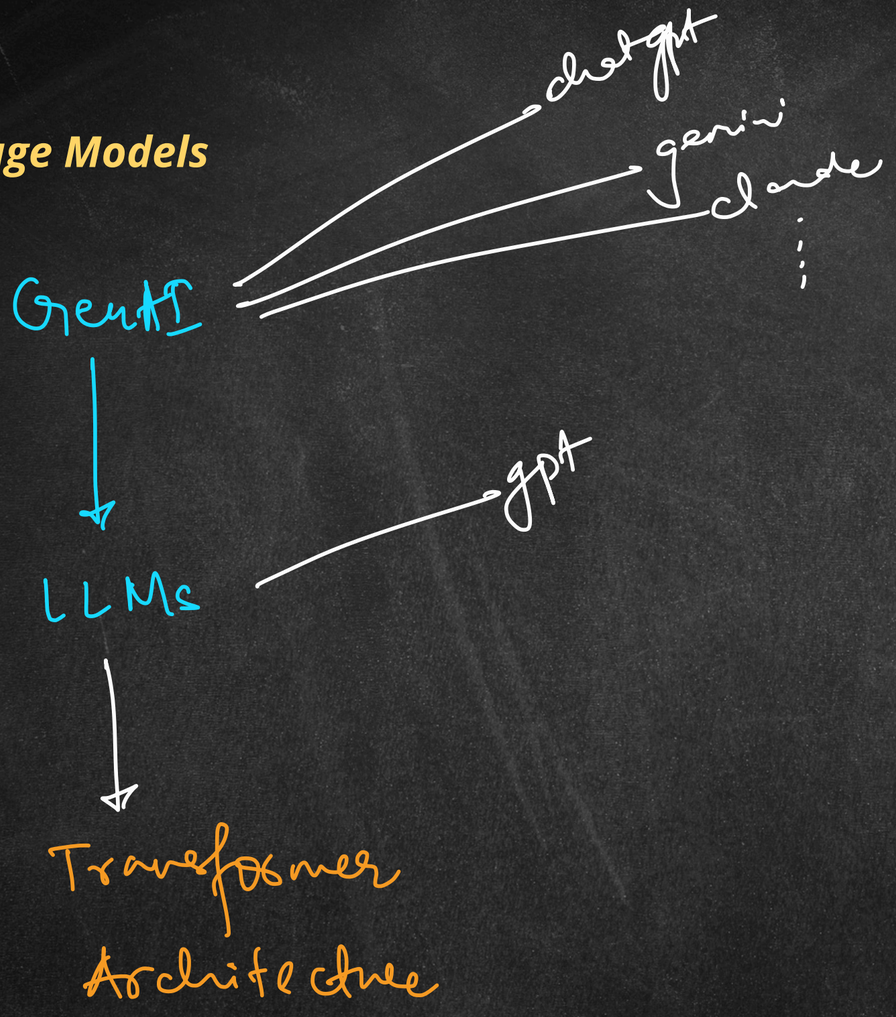
Ans: Python is a great language

GenAI → text / languages → LLMs

NLP

LLMs

Large Language Models



APNA COLLEGE

Transformers

NN arch.

2017 ⇒ "Attention is all you need"

Attention ⇒ parallelly

Sequence → text

"I like Python"

RNN, LSTM, CNN

step by step (sequentially)

sequence models

Transformers

Embedding → RNN

word embeddings → vectors

text → "I like Python" → numbs

I, like, Python

[0.1, 0.7] [0.32, 0.41] [0.8, 0.2]

features

Is it a language? proj

Is it an object?

GPT-3 → 12k+ → 12,288 dimensions in embeddings

APNA COLLEGE

Transformers

Static vs Contextual Embedding

I will read a book.

I will book movie tickets.

Word2Vec

↓

2 layer NN

↓

Static embedding

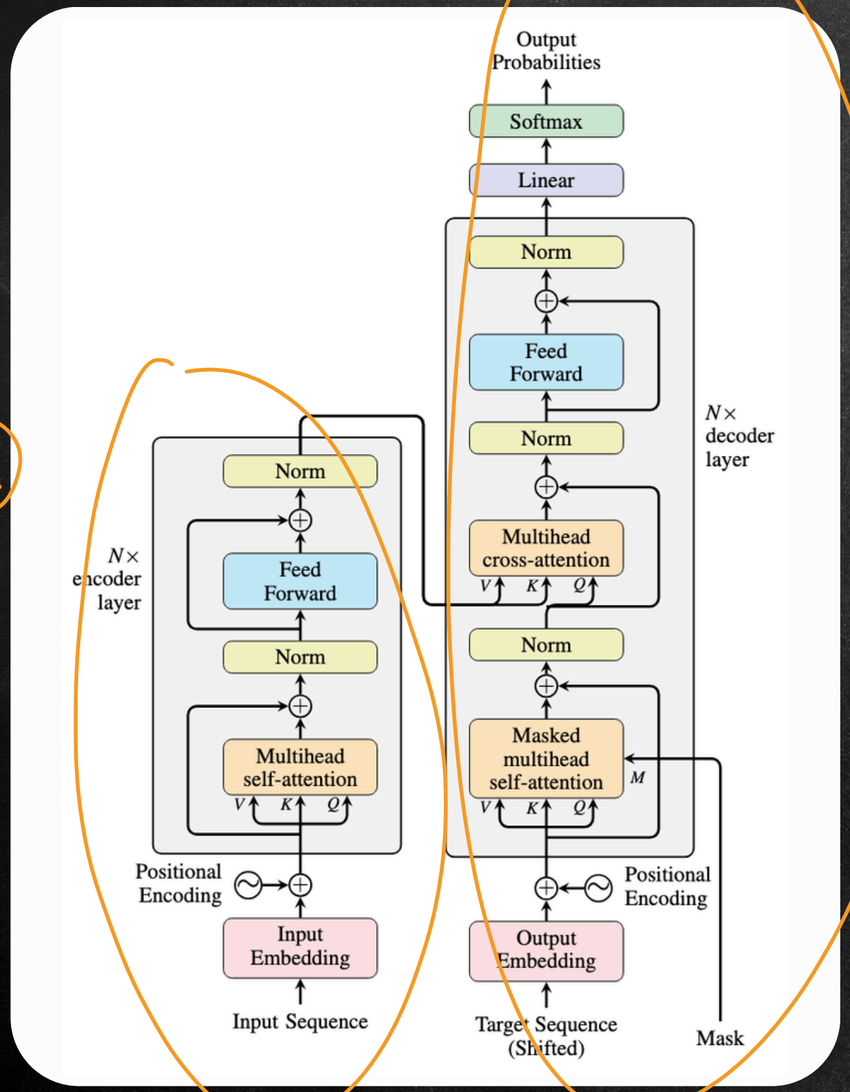
book → [0.32, 0.5, 0.01]

Python → snake

↳ prog. language

Transformers

Architecture



(1) encoder understand

(2) decoder generate

"I like Python" ⇒ contextual embeddings

inp sequence

I

liked

Python

(1) summarize

(2) sentiment analysis

(3) text classification

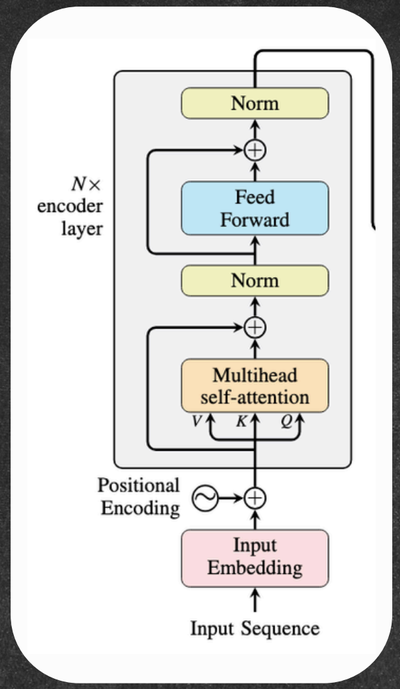
(4) speech translation

① Tokenization

② Contextual embeddings

③ pass to the decoder

"I like Python"



Encoder

"I like Python"

[I, like, Python]

0.1	0.5	0.9
0.3	0.8	0.2
....	....	....
0.0	0.2	0.3

Static embedding ⟶ contextual

+

Vocab

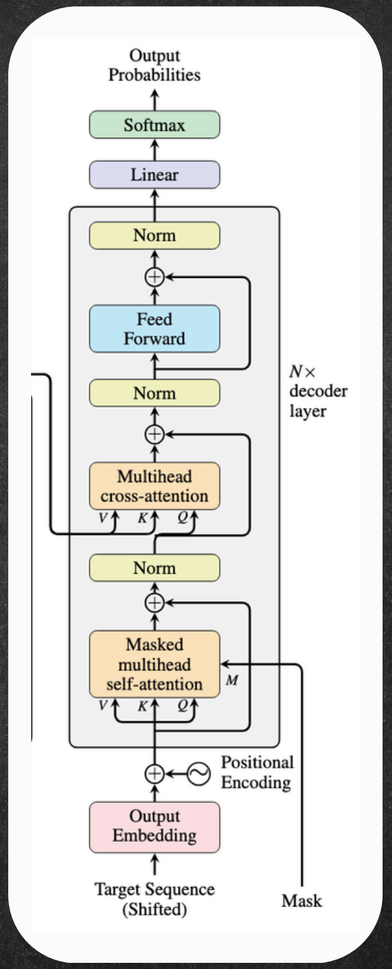
APNA COLLEGE

"I like Python"

[I, like, Python]

0.1	0.5	0.9
0.3	0.8	0.2
....	....	....
0.0	0.2	0.3

generate o/p one at a time



Decoder

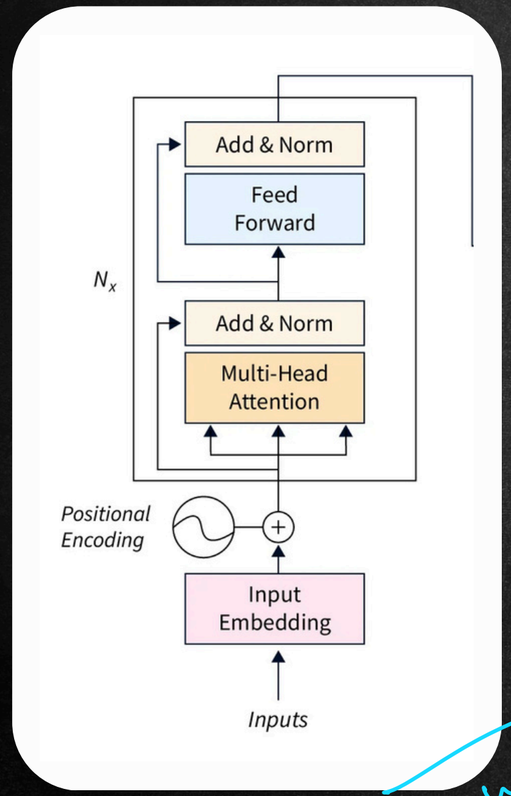
<START>
मुझे
पाइथन
पसंद
है
<EOS> end of sequence

मुझे → 70%
मैं → 10%
वह → 0.2%
यहाँ → 0.05%

APNA COLLEGE

Different Tranformer Architectures

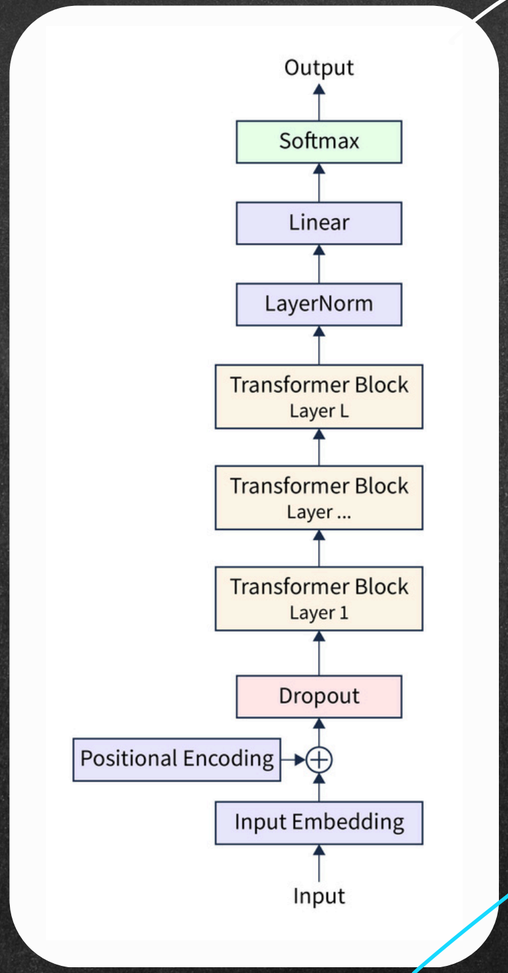
- text classification
- Sentiment analysis



understanding

BERT

encoder - only transformer arch.



text generation

GPT / Claude

decoder - only transformer arch.

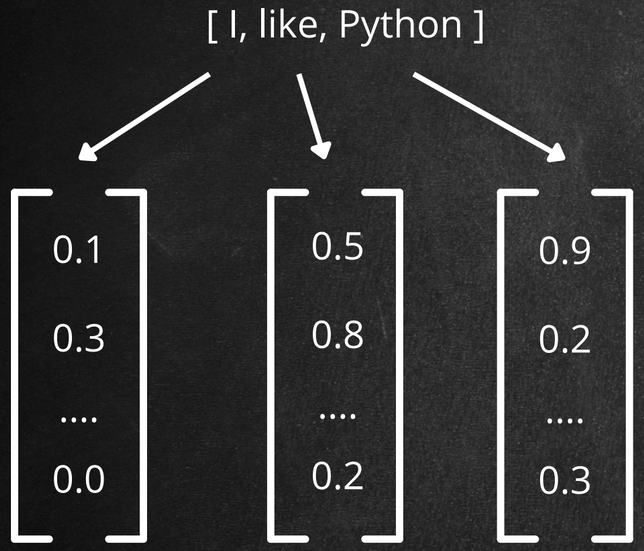
① conversation

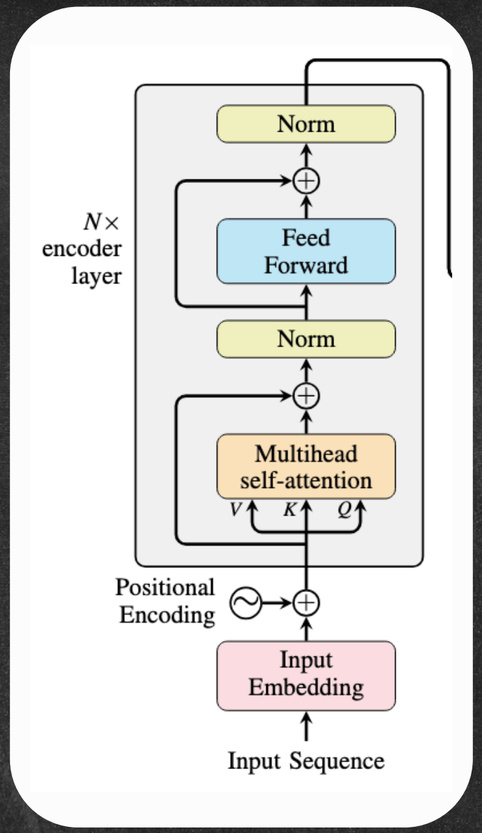
② easier to train

token by token

Encoder

Positional Encoding





I like Python

Python like I

RNN → sequentially

I, like, Python

Transformers

↓

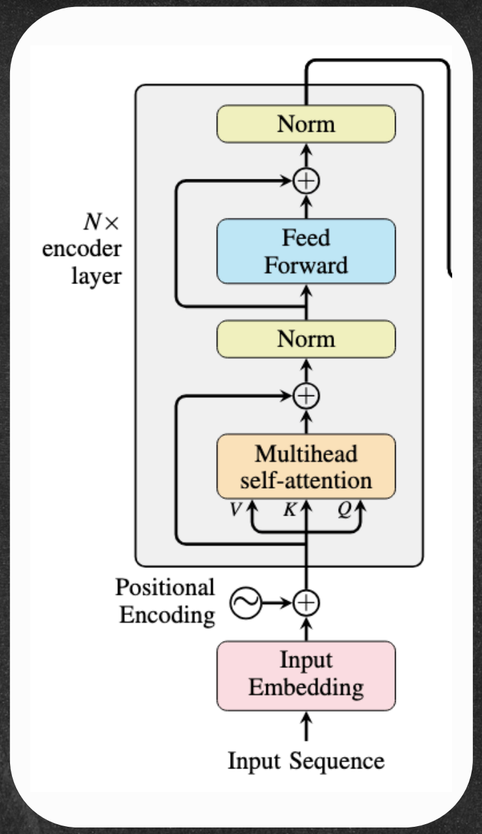
Parallelization

- training faster
- performance ↑

APNA COLLEGE

Encoder

Positional Encoding



final o/p = I/p embedding + Position embeddings

$$PE(pos, 2i) = \sin\left(pos / 10{,}000^{2i/d_{model}}\right)$$

$$PE(pos, 2i+1) = \cos\left(pos / 10{,}000^{2i/d_{model}}\right)$$

[I, like, Python]

$$\begin{bmatrix} 0.1 \\ 0.3 \\ .... \\ 0.0 \end{bmatrix} \quad \begin{bmatrix} 0.5 \\ 0.8 \\ .... \\ 0.2 \end{bmatrix} \quad \begin{bmatrix} 0.9 \\ 0.2 \\ .... \\ 0.3 \end{bmatrix}$$

$$+ \quad + \quad +$$

$$\begin{bmatrix} 0.5 \\ 0.1 \\ \dots \\ 0.3 \end{bmatrix} \quad \begin{bmatrix} \\ \\ \\ \end{bmatrix} \quad \begin{bmatrix} \\ \\ \\ \end{bmatrix}$$

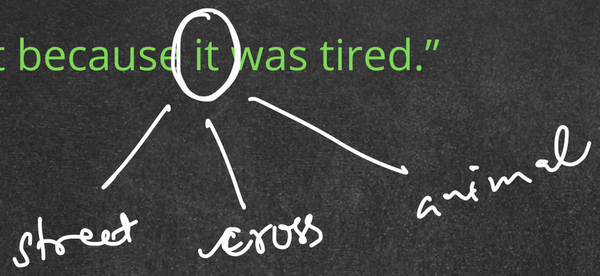
$$\begin{bmatrix} 0.6 \\ 0.4 \\ \dots \\ 0.3 \end{bmatrix}$$

Attention ⇒ focus

what/who should it pay attention to?

Attention is all you need.

"The animal didn't cross the street because it was tired."

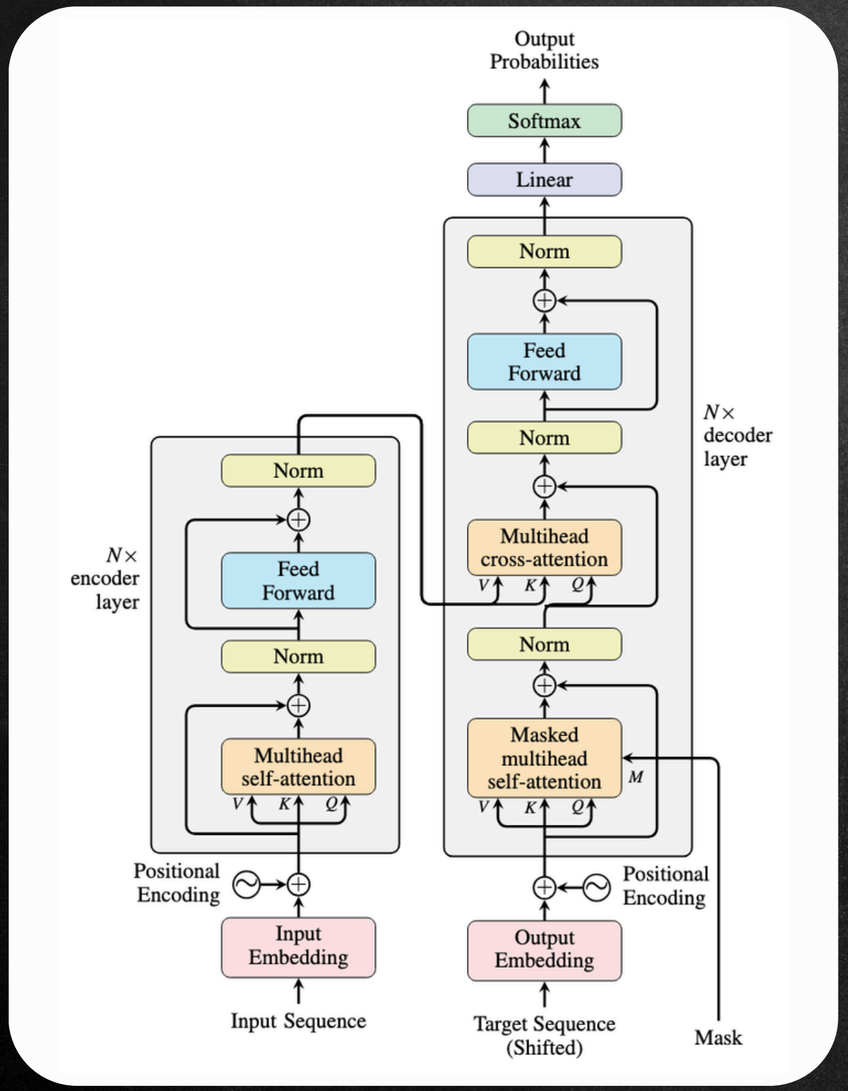


mapping

Q → K, V

Attention

Attention is all you need.



Attention

Attention is all you need.

Teacher → Transformer

APNA COLLEGE

Query (Q): what is the capital of India?

R: geography (Key) → V: capital of India — strong match = 0.8

S: math (Key) → V: 2+2=4 — low = 0.05

G: history (Key) → V: Independence/Constitution of India — med. = 0.1

3 matrices

Q, K, V

attention score ⇒ scaled dot product

Attention (Q, K, V)

Attention

Attention is all you need.

input seq → n token

d → dimension of model

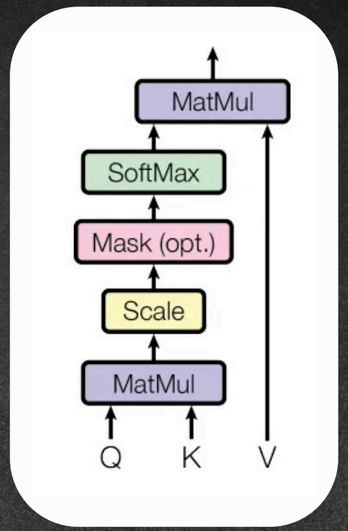
① $X = \begin{bmatrix} 0.3 \\ 0.4 \\ \cdots \\ 0.1 \end{bmatrix} \Rightarrow n \times d$

$Q = X \cdot W_Q \rightarrow d \times d_k$

$K = X \cdot W_K \rightarrow d \times d_k$

$V = X \cdot W_V \rightarrow d \times d_v$

↳ weight matrices



Scaled Dot Product Attention

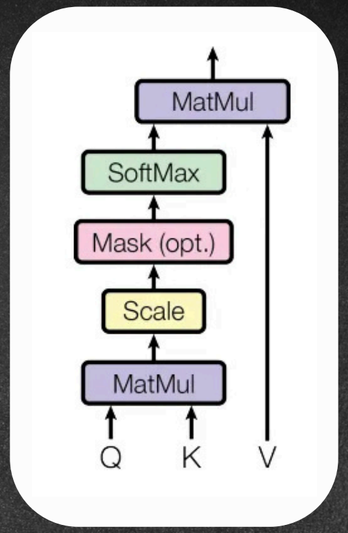
⇓

Attention(Q, K, V)

APNA COLLEGE

Attention

Attention is all you need.



geography — 0.05
maths → 0.95

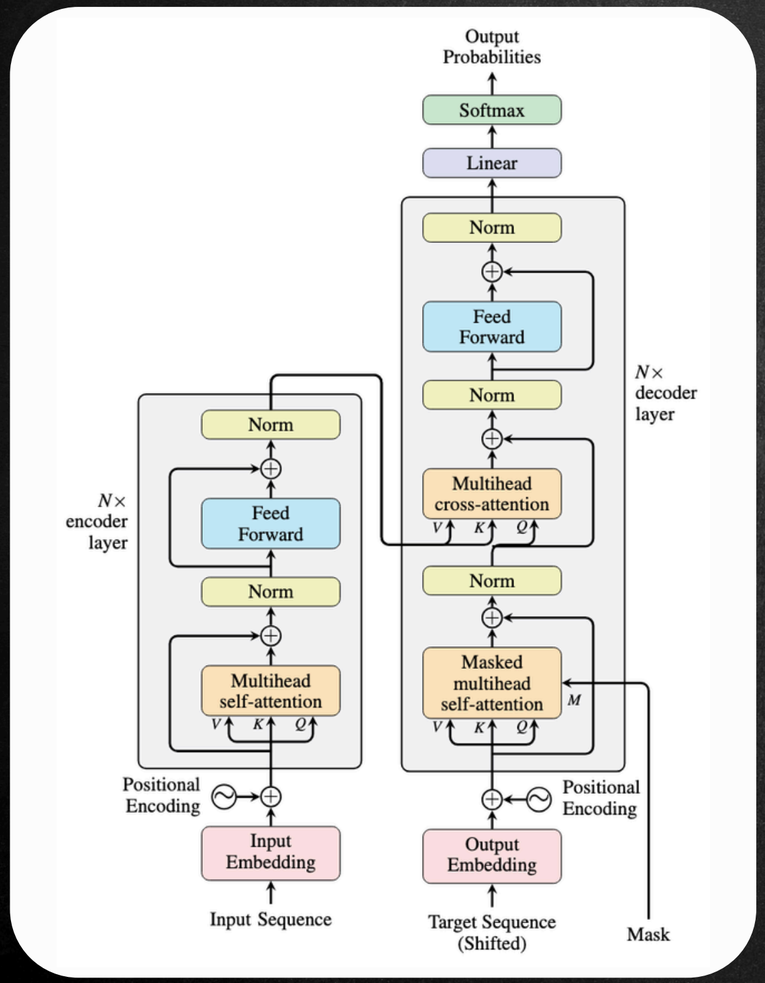
Q, K, V

(dot product)

1. similarity score ⇒ $Q \cdot K^T$
2. scaling ⇒ $\frac{Q \cdot K^T}{\sqrt{d_k}}$
3. softmax ⇒ $\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)$
   ↳ probabilities
4. $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$

Attention

Self-Attention



(SDP) scaled dot product → self-Attention → contextual embeddings

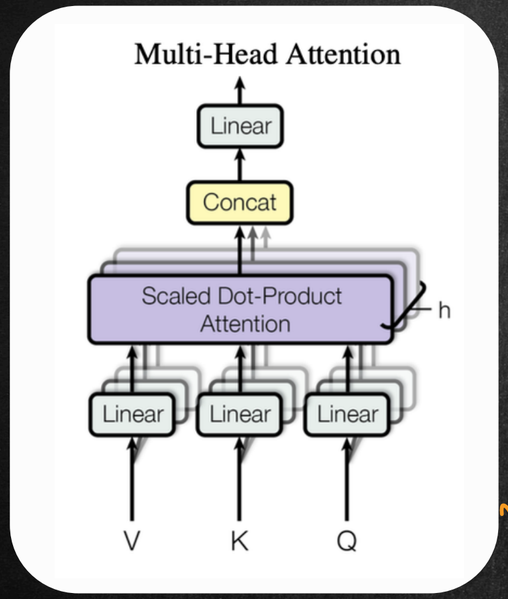
single head self-attention

i/p embedd → positional encodig → SDP → contextual embedd

APNA COLLEGE

Attention

Multi-Head Attention



multiple heads

"I like python." → i/p embed → positional modify → AH1 → CE1, AH2 → CE2, AH3 → CE3, ...

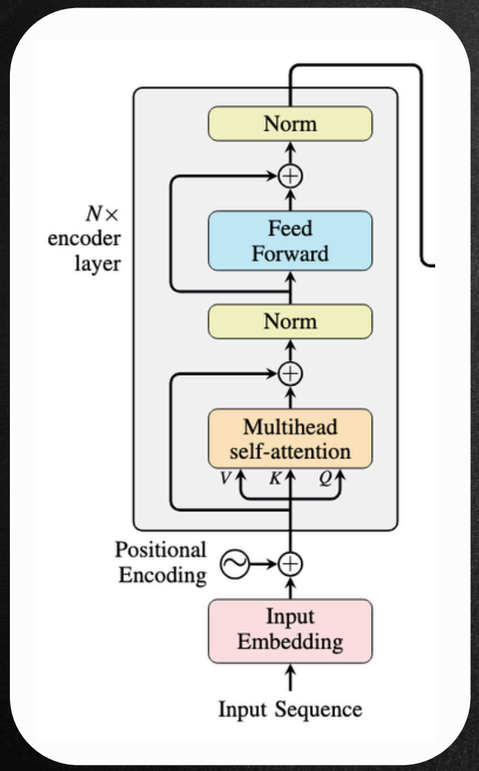
grammar

meaning

1. self-attention
2. cross-attention
3. masked attention

Encoder

Residual Connection



raw inp → directly output connect

Add + Norm

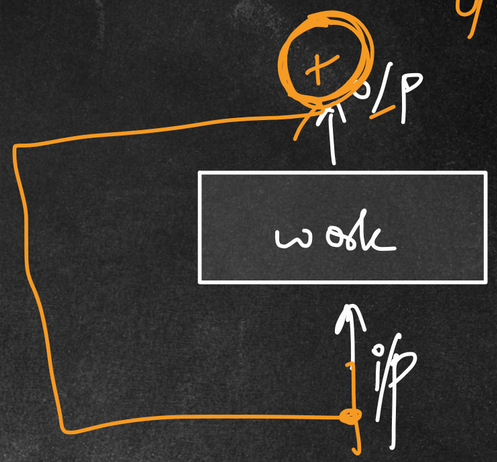
① vanishing gradient

② distorted info. in NN

residual connection

⇓

preserve original



APNA COLLEGE

Normalization

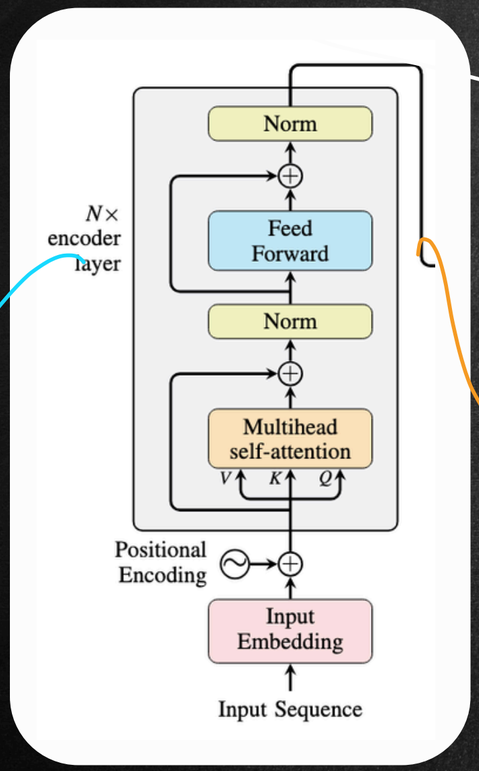
faster convergence

⇓

stabilize training

Encoder

FNN → adds non-linearity ⇓ " complexity → more enriched understanding



→ contextual embedd.

gpt2 → N = 48

gpt3 → N = 96

① gather infor.

② think deeply info.

→ imp features ↑↑

→ irrelevant info ↓↓

APNA COLLEGE

Decoder

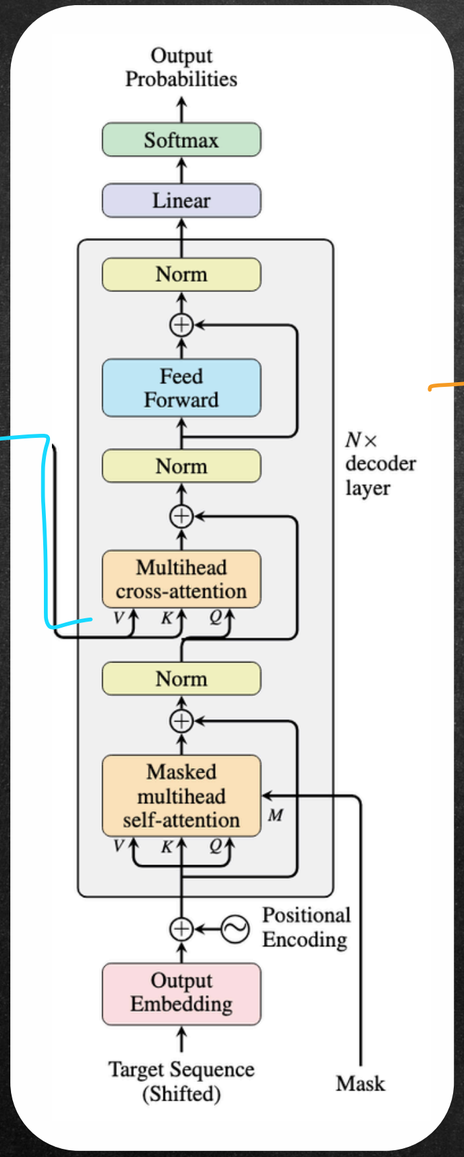
Cross-Attention

CROSS - ATTENTION

↓

attention btw 2 diff sequences

APNA COLLEGE



contextual embedding

I → ce1

like → ce2

Python → ce3

"I like Python"

<START>

मुझे

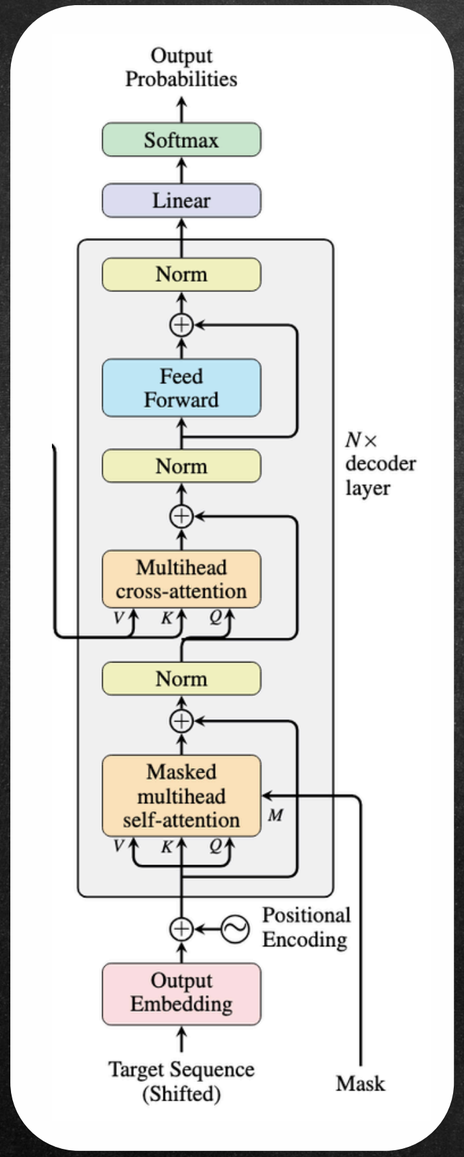
Q: next word?

पायथन → 80%

Decoder

Masked Attention

⇒ Apples and oranges are fruits.



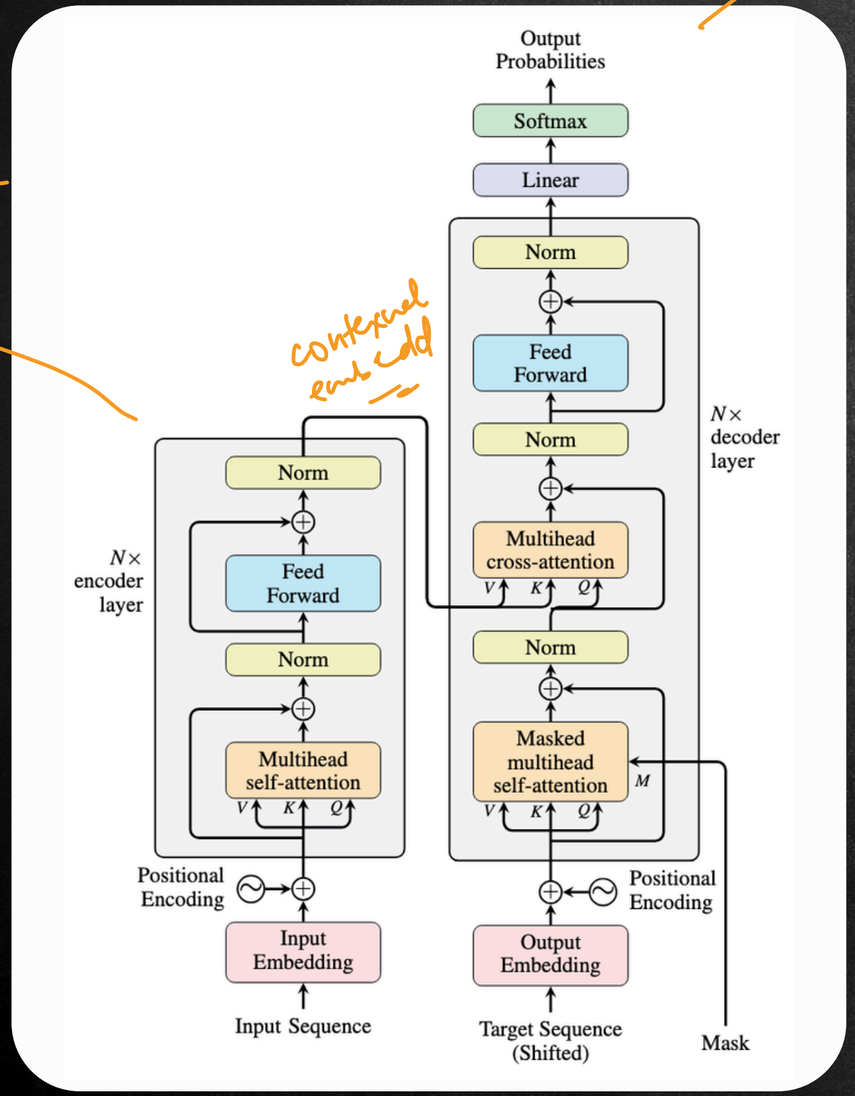
Training ⇒ cheating ✗

{ multi. head self-attention + mask

APNA COLLEGE

Transformer

Architecture Summary



generation word by word

understand

context embedding

बुरा — 2%

पसंद — 72%

है — 10%

मुझे पढ़ना ... Q: — ? पसंद है।

Pre-trained Transformers

BERT
GPT-3
T5
Text to Text Transfer Transformer

APNA COLLEGE

already trained on huge datasets for general tasks

⇓ fine tune

SOTA AI models

self-training

1. small dataset → poor generalization / overfitting ⇒ better generalization
2. time ↓ → save
3. money ↓ → billions of params

Pre-trained Transformers

APNA COLLEGE

- Hugging Face
- PyTorch Hub
- TensorFlow Hub
- OpenAI APIs etc.